

クラウドLLMとローカルLLMをプライバシー境界で切り分ける呼び出しフレームワーク調査

エグゼクティブサマリー

本調査の結論を先に述べると、NBAccessに最も近い発想は、単なる「モデル選択」ではなく、**機密データの实体をどこまで信頼境界の内側に残し、どの時点で、どの単位で、どの経路へ出すかを明示的に制御すること**にあります。公開一次資料を比較すると、この要件を満たす実装は大きく四系統に分かれます。すなわち、**インプロセスの安全カーネル型**（NBAccess）、**OpenAI互換のAIゲートウェイ型**（LiteLLM、Kong、Portkey、Azure API Management AI Gateway）、**アプリ内オーケストレーションSDK型**（Haystack、LangChain/LangGraph、Semantic Kernel、LlamaIndex）、そして**ガバナンスを前面に出したメッシュ型**（Dataiku LLM Mesh）です。これらは同じ「ルーティング」を名乗っていても、保護対象、強制点、監査面、そして失敗時の挙動がかなり異なります。 ¹

最も強いプライバシー境界を作りやすいのは、アプリ外部に**独立したゲートウェイ/ポリシー面**を設ける方式です。Kong AI Gateway、Portkey、Azure API Management AI Gateway、LiteLLM Proxy は、認証情報の集約、接続先の抽象化、監査、レート制御、フォールバック、モデル許可制御を一箇所に寄せられるため、アプリから「直接クラウドへ送ってしまう」事故を減らしやすい構造です。一方、Haystack、LangChain、Semantic Kernel、LlamaIndex は**柔軟性が高い反面、強制力は設計次第**であり、同一プロセス内で分類・マスキング・ルーティングをやる以上、ミドルウェアを通る前に秘密が別のログやメモリ断面に露出する設計ミスを起こしやすい、という限界があります。 ²

NBAccessそのものは、今回の調査対象の中でも特に「機密データの实体はNBAccessだけが扱う」「外部LLMはリソースへ直接アクセスしない」「LLMは式を生成し、実行前に検証される」という不変条件を明示しており、さらに `NBRouteDecision`、`NBAuthorize`、`NBValidateHeldExpr` のようなAPIで、アクセスレベル・プロバイダ・実行効果クラスをまたいだ制御点を提供しています。この意味で、NBAccessは“ルーター”というより**信頼境界付きの実行カーネル**に近く、一般的なSDKやゲートウェイより思想が一段深いです。類似の方向性に近いのは、KongのAI Proxy + AI PII Sanitizer / Prompt Guard、LiteLLMのAccess Groups + Guardrails + Presidio連携、PortkeyのPII Redaction + `custom_host`、Dataikuのconnection-level / usage-time guardrails です。 ³

ただし、**形式的なプライバシー保証**という観点では、今回の主要製品群はほぼすべてが**境界制御・匿名化・最小化・分離配置**に依存しており、**ネイティブな differential privacy、SMPC、TEEベース機密推論**をコア機能として提供してはおりません。これは重要なギャップです。公開一次資料および関連研究を合わせてみると、実運用の主流は依然として「ローカル保持」「送信前マスキング」「外向きゲートウェイ」「監査」の組み合わせであり、より強い保証はLLM Access ShieldやAloePri、TEE/DP系研究の領域にあります。したがって、“**ローカルにルーティングできる**”ことと、“**形式的にプライベートである**”ことは**同義ではない**、という点を強調しておく必要があります。 ⁴

実務上の推奨は明確です。**最小構成で始めるなら LiteLLM または Kong、厳格な企業統制なら Portkey・Azure APIM AI Gateway・Dataiku LLM Mesh、アプリケーション内で細かい研究開発をしたいなら Haystack / Semantic Kernel / LangChain / LlamaIndex、そしてNBAccess的な強い境界を目指すなら“分類→変換→認可→ルーティング→監査”を独立面として切り出すべきです。**とくに、「**ローカル失敗時にクラウドへ自動フォールバックする**」挙動は、機密要求では既定で禁止しないと、もっとも見落とされやすい境界破りになります。 ⁵

調査範囲と評価軸

このレポートでは、オープンソースと商用のフレームワーク、ミドルウェア、オーケストレーション層、SDK、プラグインを対象に、リクエスト単位またはデータ単位で、クラウドLLMとローカルLLMを切り替えられるかを主眼に調べました。明示的に対象としたのは、NBAccess、LiteLLM、Haystack、LangChain/LangGraph、Semantic Kernel、LlamaIndex、Kong AI Gateway、Portkey、Azure API Management AI Gateway、Dataiku LLM Mesh です。これらはすべて、一次資料上で少なくとも「多モデル/多プロバイダ接続」「ルーティング/フォールバック/ミドルウェア/フィルタ/ガードレール/ポリシー」のいずれかを確認でき、かつローカル系またはOpenAI互換の私設エンドポイントと、クラウド系モデルを同じ設計面へ載せられるものです。

ここでいうプライバシー境界は、単なるAPI ルータではなく、次の要素のどこまでを備えるかで判断しました。すなわち、アーキテクチャ、データフロー、ポリシーモデル、強制点、信頼境界、暗号化/セキュアトランスポート、オンデバイス処理、DP/SMPC/TEE などの強いプライバシー技術、監査/ログ、性能・スケーラビリティ、配置形態、統合API、対応LLM、ライセンス、成熟度とサポートです。評価観点としては、NIST AI RMF の Govern / Map / Measure / Manage の発想と、OWASP の LLM/GenAI リスク整理を下敷きしつつ、「境界をどこに置くと実務で事故が減るか」を重視しました。

今回、予算制約とクラウド事業者の制約は指定されていないため、AWS・Azure・GCP・任意のオンプレ/ローカル実行を横断比較しています。また、統一条件の性能ベンチマークは指定されていないため、レイテンシ/スループットはベンダー横断の単純比較ではなく、公開一次資料に見える構成要素——たとえば「追加ホップの有無」「前処理/後処理ガード」「同期/非同期」「自己ホスト可否」——から相対評価しています。これは、プロキシやガードレールが1段入るだけでも遅延特性が変わる一方、公開資料側では必ずしも同条件比較が用意されていないためです。

実務上もっとも重要なのは、境界の位置です。大まかには、
アプリ内部の境界 (NBAccess、Haystack、LangChain、Semantic Kernel、LlamaIndex)
アプリ外部のゲートウェイ境界 (LiteLLM、Kong、Portkey、Azure APIM)
統制面まで含むガバナンス境界 (Dataiku)
に分かれます。一般に、アプリ外に出した方が強制力は上がり、アプリ内に置いた方が柔軟性は上がる、というトレードオフがあります。

フレームワーク比較表

比較マトリクス

フレームワーク	区分	プライバシー・ルーティング適性	ポリシーモデルと強制点	信頼境界とセキュリティ特性	ローカル/クラウド対応と主なLLM
NBAccess	OSS / インプロセス安全カーネル	◎ ネイティブ。 NBRouteDecision、NBAuthorize、NBValidateHeldExpr があり、AccessLevel / Provider / EffectClass を跨いで判定できる。	実行前の式検証、semantic API 越しアクセス、外部LLM直接アクセス禁止。	「機密データ実体はNBAccessだけが扱う」と不変条件を明示。Windows 11 + Mathematica 前提。DP/SMPC の明示はなし。	repo例ではlmstudioとCloudLLMの双方を前提。

フレームワーク	区分	プライバシー・ルーティング適性	ポリシーモデルと強制点	信頼境界とセキュリティ特性	ローカル/クラウド対応と主なLLM
LiteLLM	OSS + 商用 / OpenAI 互換 ゲートウェイ	◎ 強い 。Access Groups、Virtual Keys、fallbacks、router settings、guardrails、Presidio 連携で per-request 制御がしやすい。	鍵・チーム・モデル群・guardrail hook が強制点。フォールバックとアクセス制御を Key/Team 単位で調整可能。	境界はプロキシ外側。PII masking は Presidio / Purview 等と統合。DP/SMPC は一次資料で確認できず。	OpenAI、Anthropic、Bedrock、Vertex、Azure OpenAI、Ollama、OpenAI-compatible endpoints など広範。
Haystack	OSS / オークレストレーション	○ 合成可能 。ConditionalRouter、MetadataRouter、Transformers系Router、LLMMessagesRouter で per-request / per-data 分岐を構成できる。	ルータ部品が強制点。前段で分類し、OpenAIChatGenerator / OllamaChatGenerator / FallbackChatGenerator へ分岐可能。	同一アプリ内境界なので強制力は設計次第。ログ/トレースは OpenTelemetry・Datadog・Langfuse・LoggingTracer に対応。	OpenAI 系・Azure OpenAI・Ollama・NVIDIA NIM などの generator/integration を備える。
LangChain / LangGraph	OSS / SDK・エージェント基盤	○ 合成可能 。middleware で PII detection、guardrails、fallback、状態依存分岐。LangGraph は条件分岐グラフを明示的に組める。	middleware と graph edge が強制点。pre/post model hook で PII redaction や output guard を挿入できる。	同一プロセス境界。観測面は LangSmith で強いが、強制面は設計依存。	OpenAI、Anthropic、Azure、Hugging Face ほか統一インタフェース。Ollama integration あり。
Semantic Kernel	OSS / SDK	○ 合成可能 。複数 AI service を <code>serviceId</code> で登録し、PromptExecutionSettings/filters でリクエストごとに選択できる。	<code>serviceId</code> 、filters、kernel 引数が主な強制点。関数呼び出し・プロンプト描画の両方にフィルタを入れられる。	同一アプリ内境界だが、フィルタは enterprise readiness を明示。	Azure OpenAI、OpenAI、Google、Mistral、Hugging Face、Azure AI Inference、Ollama、ONNX、Bedrock など。

フレームワーク	区分	プライバシー・ルーティング適性	ポリシーモデルと強制点	信頼境界とセキュリティ特性	ローカル/クラウド対応と主なLLM
LlamaIndex	OSS / データ・RAG・エージェント基盤	○ 合成可能 。Router/RouterRetriever/RouterQueryEngineにより問い合わせの行き先を選べる。	selector / router / workflow step が強制点。	同一アプリ内境界。 observability / instrumentation は強いが、 privacy routing は開発者実装が前提。	OpenAI、Anthropic、Mistral、DeepSeek、Hugging Face など多数に加え、Ollama を使った local LLM も公式に案内。
Kong AI Gateway	OSS + 商用 / AIゲートウェイ・プラグイン	◎ 強い 。AI Proxy / Proxy Advanced、Prompt Guard、Semantic Prompt Guard、PII Sanitizer、外部安全系プラグインで境界をゲートウェイ外へ出せる。	プラグインチェーンが強制点。allow/deny、semantic guard、PII sanitize、audit logs を独立面で適用。	provider-agnostic API。Ollama を含むローカルLLMとOpenAI/Vertex等のクラウドを同じ面で扱える。 vault/TLS/自前プラグインも使いやすい。	OpenAI、Anthropic、Azure AI、Vertex、Databricks、Ollama ほか。
Portkey	商用 / AIゲートウェイ	◎ 強い 。Configs による routing rules、PII Redaction、custom_host による local/private model、監査ログ、OTel export、hybrid deployment が揃う。	Gateway config と guardrail が強制点。	control plane / data plane 分離、hybrid では data plane を自社環境に置き、sensitive data stays within your infrastructure と明示。	OpenAI互換プロジェクト全般、250+ providers、custom_host で private model ヘルディング可能。
Azure API Management AI Gateway	商用 / エンタープライズ API ゲートウェイ	○～◎。OpenAI-compatible / pass-through LLM endpoint の仲介、backend pool、self-hosted gateway、ポリシー適用で境界を強められる。	APIM policy、backend、self-hosted gateway が強制点。	境界は明確にゲートウェイ外。多環境モデル管理に強い。DLP/PII 自体は隣接サービスやカスタムポリシー併用が前提。	OpenAI-compatible、Anthropic Messages、Vertex AI API、Bedrock 等のスキーマ/エンドポイントを管理可能。

フレームワーク	区分	プライバシー・ルーティング適性	ポリシーモデルと強制点	信頼境界とセキュリティ特性	ローカル/クラウド対応と主なLLM
Dataiku LLM Mesh	商用 / ガバナンス・メッシュ	◎ 強い 。connection-level guardrails と usage-time guardrails、approved services/datasets、quotas、monitoring を統合。	LLM connection / Mesh / guardrails が強制点。	self-managed / on-prem/cloud tenant 配置が可能で、Cloud Stacks は “everything is deployed in your cloud tenant” と明示。 PrivateLink/PSC と相性が良い。	多数の hosted LLM API に加え、custom internal gateway、NVIDIA NIM、local Hugging Face 系を統合可能。

評価基準に照らした短評

セキュリティ/プライバシー保証で最も高得点なのは、**NBAccess**、**Kong**、**Portkey**、**Dataiku**、**Azure APIM**です。理由は、ポリシー強制点がアプリ本体から分離されているか、少なくともNBAccessのように「外部 LLMは直接アクセスしない」という不変条件が明示されているためです。反対に、LangChain・Haystack・Semantic Kernel・LlamaIndex は、構成自由度は高いものの、**強制点がアプリ内に埋め込まれる**ので、実装ガバナンスが弱い組織では境界が崩れやすいです。 ²⁰

統合容易性で最も扱いやすいのは、**LiteLLM**、**Portkey**、**Kong**、**Azure APIM**です。いずれも OpenAI 互換ないし provider-agnostic な境界面を持ち、既存アプリ側の変更を最小化しやすいからです。アプリを書き換えずにベースURLを差し替えるだけで導入しやすい Portkey と LiteLLM、既存API管理組織に乗せやすい Kong と Azure APIM は、とくに導入障壁が低い側に入ります。 ²¹

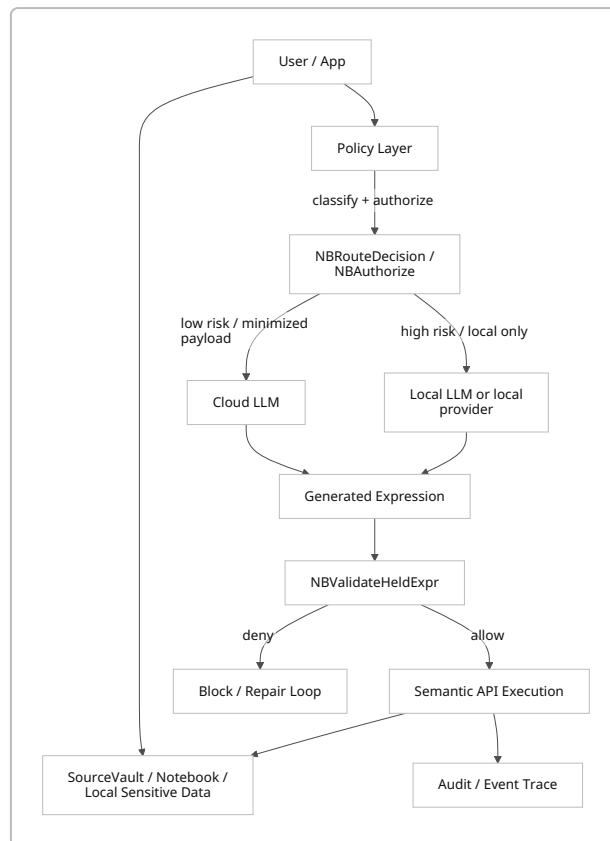
性能・スケーラビリティは、実環境差が大きく、公開資料だけで断定的順位づけはできません。ただし、一般論としては、**ゲートウェイ + ガードレール + 外部PIIサービス + 完全トレース**のようにホップが増えるほど遅延は上がりやすく、逆に**ローカル推論**はネットワーク遅延を抑えられる一方で、計算資源制約でスループットが頭打ちになりやすいです。したがって、高秘匿・低遅延が必要な箇所は「小型ローカルモデル + 厳密ポリシー」、高品質生成は「匿名化後クラウド」という分業が現実的です。各製品の observability / tracing 機能は、まさにこのトレードオフを計測するために重要です。 ²²

コストは、短期では SaaS ゲートウェイやクラウドモデルの方が安く見えやすく、長期では**高機密トラフィックをどれだけローカル/専有環境へ寄せるか**で逆転します。Dataiku や Portkey、Kong、Azure APIM はガバナンス・監査・運用費をまとめて吸収しやすい一方、OSS の LiteLLM + Haystack / SK / LangChain 構成はライセンス費を抑えられます。その代わり、境界設計・監査・運用の責任は自社に残ります。 ²³

代表アーキテクチャ

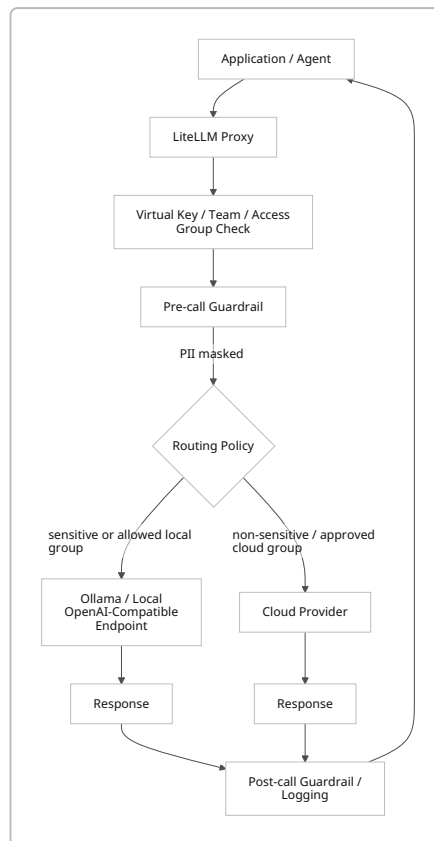
NBAccess 型

NBAccess は、LLM を「直接データに触る主体」にせず、**式提案者**に限定し、**Notebook / source / claims / bundles** への**実体アクセスは semantic API と vault 側に残す**設計です。これはクラウドかローカルかの選択以前に、“LLMが何を見られるか”と“LLMが何を実行できるか”を分けるアーキテクチャです。NBAccess 由来の設計思想をそのまま別言語へ移すなら、この分離を最優先にすべきです。 ²⁴



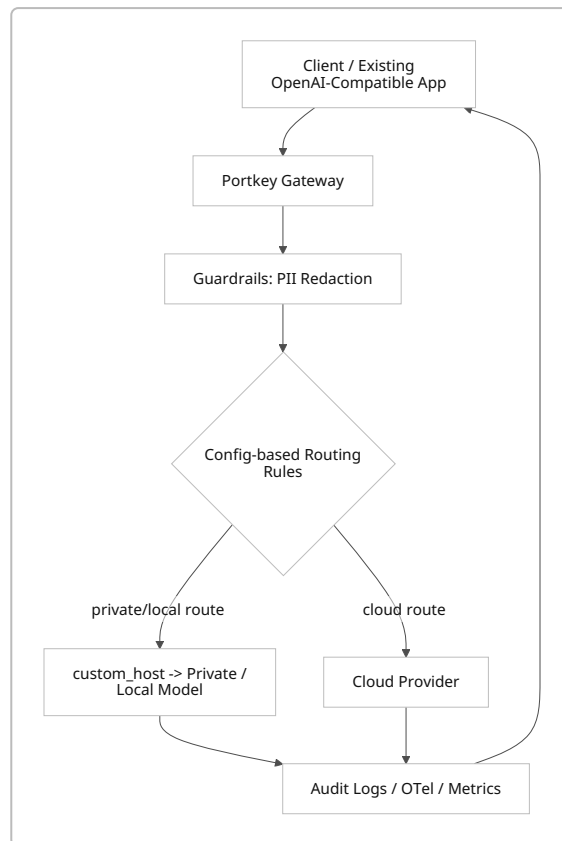
LiteLLM ゲートウェイ型

LiteLLM は、アプリから見れば単一の OpenAI 互換入口でありながら、その裏側で Ollama のようなローカル実行と OpenAI / Anthropic / Bedrock / Vertex のようなクラウド実行を切り替えられます。Privacy routing の観点では、Access Groups で誰が何モデルを使えるか、Guardrail で何を出せるか、Presidio/ Purview 連携で何を伏せるかを、一つの egress 面にまとめられる点が重要です。 25



Portkey ゲートウェイ型

Portkey は、**構成JSONで routing rules を定義しつつ、PII redaction と custom_host による private model ルーティング**を同じ製品面で扱えるのが特徴です。さらに hybrid deployment では、**AI Gateway と data plane を自社環境に置き、sensitive LLM data を自社インフラ内に残す構成**が取れます。これにより、「中央集権的なポリシー管理」と「データ平面の自社保持」を両立しやすいです。 26



実装例と設計パターン

短いコードと設定例

以下の例は、公式APIやドキュメントに沿って簡略化した“実装パターン例”です。URLは直接書かず、対応するリポジトリや一次資料を出典で示しています。NBAccess、LiteLLM、Haystack、Semantic Kernel は、いずれもこの程度の小さな断片から privacy routing を始められます。 ²⁷

NBAccess 風の手前判定

```

(* 機密度が高い場合は local provider を優先し、CloudLLM は禁止 *)
policy = <|"AccessLevel" -> 0.9, "Provider" -> "lmstudio"|>;

decision = NBRouteDecision[policy["AccessLevel"]];
authz = NBAuthorize[policy];

If[decision["Route"] === "LocalLLM" && TrueQ[authz["Authorized"]],
  (* local provider only *),
  (* block or repair *)
]

```

この形の本質は、モデル呼び出しの前に認可があり、さらに生成物の実行前に別の検証があることです。NBAccess の repo では、NBRouteDecision、NBValidateHeldExpr、NBAuthorize の使用例が明示されています。 ²⁸

LiteLLM で「高機密はローカル、一般はクラウド」

```
model_list:
- model_name: local-safe
  litellm_params:
    model: ollama/llama3.1
- model_name: cloud-general
  litellm_params:
    model: openai/gpt-4o-mini

litellm_settings:
  fallbacks:
    - {"cloud-general": ["local-safe"]}
```

ポイント:

- # 1) 機密ワークロード用キーには local-safe だけを許可
- # 2) 一般ワークロード用キーには cloud-general を許可
- # 3) Guardrails で送信前マスキング

実運用では、機密フローに“cloud-general への逆フォールバック”を入れないことが大事です。LiteLLM は Access Groups、Virtual Keys、router settings、fallbacks、Presidio/Purview 系 guardrails を備えています。

29

Haystack でデータ分類後に OpenAI と Ollama を分岐

```
from haystack import Pipeline
from haystack.components.routers import ConditionalRouter
from haystack.components.generators.chat import OpenAIChatGenerator
from haystack_integrations.components.generators.ollama import OllamaChatGenerator

router = ConditionalRouter(routes=[
    {
        "condition": "{{ sensitivity == 'high' }}",
        "output": "{{ messages }}",
        "output_name": "local_msgs",
        "output_type": list,
    },
    {
        "condition": "{{ sensitivity != 'high' }}",
        "output": "{{ messages }}",
        "output_name": "cloud_msgs",
        "output_type": list,
    },
])

local_llm = OllamaChatGenerator(model="qwen3:8b")
cloud_llm = OpenAIChatGenerator(model="gpt-4o-mini")

pipe = Pipeline()
```

```
pipe.add_component("router", router)
pipe.add_component("local_llm", local_llm)
pipe.add_component("cloud_llm", cloud_llm)
```

Haystack の利点は、**分類器とルータと生成器が明確に部品化されている**ことです。欠点は、**強制点がアプリ内にあるので、ログや周辺コードまで含めて自前できれいに設計しないと境界があいまいになる**ことです。

30

Semantic Kernel で serviceId を切り替える

```
# 概念例: local / cloud のサービスを両方登録し、
# request ごとに service_id を選ぶ

from semantic_kernel.connectors.ai import PromptExecutionSettings

settings = PromptExecutionSettings(
    service_id="ollama_local" if sensitivity == "high" else "azure_openai_prod"
)
```

Semantic Kernel は、**複数のAI service を同時に登録して service_id で狙い撃ち**できるため、privacy routing を比較的きれいに書けます。さらに filters で前後処理を掛けられるので、**分類→service選択→出力制約**の流れが作りやすいです。 31

推奨設計パターン

実務でうまくいきやすいのは、**モデルルーティングを“推論インフラ問題”としてではなく、“情報フロー制御問題”として扱う**ことです。NBAccess が示すように、理想形は「データに直接触る層」「モデルを呼ぶ層」「生成物を実行する層」を分離することです。アプリ内SDKしか使わない場合でも、この発想をそのまま写像すると、事故が大きく減ります。 32

第一に、**分類をシリアル化前にやる**ことです。クラウドへ送る本文を作った後で「やはり機密だった」と気付く設計では遅すぎます。Kong の AI PII Sanitizer や Portkey の PII Redaction、LiteLLM の Presidio 連携は有用ですが、それらは**最後の砦**であって、第一防衛線ではありません。データクラス分類は、アプリ内部または trusted middleware の最前段で行い、**機密本文そのものが egress 用文字列へ展開される前に経路を決める**べきです。 33

第二に、**タスク分解とデータ最小化をセットに**することです。秘密を含む検索・取得・埋め込みはローカルで行い、クラウドへ出すのは**匿名化済みの要約・ラベル・抽象化済み特徴量**だけにする、という分業です。LlamaIndex や Haystack の router / retriever 構造、Dataiku の LLM Mesh / guardrails は、この分業に向いています。 34

第三に、**フォールバックをプライバシー境界の一部として扱う**ことです。可用性設計ではフォールバックが善に見えますが、機密経路では**禁止されるべきフォールバック**があります。たとえば「ローカル高機密モデルが落ちたので、一般クラウドモデルへ迂回する」は典型的な境界破りです。LiteLLM、Kong、Portkey、Azure APIM は経路制御を中央集権化しやすいので、この“禁止される迂回”を明示的に閉じる設計に向きます。 35

第四に、**観測系と秘匿系を分離**することです。LangSmith、Haystack tracing、Portkey complete logs export、Kong audit logs は非常に有用ですが、トレース自体が新しい機密面になります。したがって、“**監査**

は残すが本文は残さない”、“保存してよいのはハッシュ・カテゴリ・trace_id まで”、“全文保存は自社閉域のみ”といった二重ルールが必要です。 36

実装チェックリスト

以下は、NBAccess 類似の privacy boundary を実装する際の、実務向けチェックリストです。NIST AI RMF と OWASP/NCSC が強調する「ガバナンス」「攻撃面」「被害最小化」の観点を、今回の製品比較に落とし込んだものです。 37

- **データ分類表**を先に作る。公開可、社内限定、個人情報、機微情報、規制対象を区別する。
- 各分類に対して、**local-only / private-endpoint-only / cloud-allowed** を決める。
- **クラウド送信前の最小化**を標準化する。PII redaction、プレースホルダ化、匿名ID化、摘要化。
- **フォールバックの許可表**を作る。特に「local → cloud」は分類ごとに deny-by-default にする。
- **強制点を一箇所に寄せる**。可能ならゲートウェイ、少なくとも共通 middleware にする。
- **監査ログの赤化対策**を入れる。本文は残さず、decision / policy / trace_id / model_id / scope を残す。
- **秘密管理**はアプリ側でなくゲートウェイ側に寄せる。Vault、秘密参照、PrivateLink、self-hosted gateway を優先する。
- **prompt injection / data exfiltration テスト**を継続的に回す。
- **レイテンシ測定**は warm/cold、local/cloud、PII sanitization on/off の軸で別々に採る。
- “**復元可能匿名化**”機能の扱いを明示する。復元は便利だが、信頼境界を再び広げる。

ギャップとリスク

最大のギャップは、すでに触れた通り、**主要フレームワークの多くが“境界制御型”であり、“形式保証型”ではない**ことです。OWASP も NCSC も、LLM 系の攻撃面を従来の入力検証だけで片付けられないと繰り返し指摘しています。NCSC は prompt injection を SQL injection 的な問題として見るべきではなく、**LLM を inherently confusable deputy とみなして被害最小化するべきだ**と述べています。これは、どれだけルーティングが賢くても、一度モデルが機密を見たら、その後の完全統制は難しいという意味です。 38

この危険は、エージェント/ツール呼び出しでさらに増幅します。2025年の研究では、**LLMエージェントが作業中に観測した個人情報を、prompt injection によって漏洩させられる**ケースが示されており、しかも既存防御では十分でない場面があります。したがって、privacy routing は「どのモデルを呼ぶか」だけでなく、**どのツールに何のスコープを与えるか、どこまでの中間生成物を別コンポーネントへ渡すか**まで含めて設計しなければなりません。 39

第二のギャップは、**フィールド単位・意味単位の分割送信**がまだ弱いことです。多くの製品は「このリクエスト全体を local へ」「このリクエスト全体を cloud へ」という粒度では強いのですが、**“検索はローカル、推論だけクラウド、ただし固有名詞は匿名化、出力復元はローカルでのみ”**のような厳密な **per-data decomposition**は、結局アプリ側の設計力に依存します。NBAccess が相対的に優れているのは、まさにこの分解を実行モデルに埋め込んでいる点です。 40

第三のギャップは、**ログ/トレースが第二の漏洩面になる**ことです。Portkey は complete logs export を、LangSmith は end-to-end traces を、Haystack は content tracing を、Kong は structured AI logs を提供します。これは運用に必須ですが、**“リクエスト本文を保護する”**という発想だけでは不十分で、**“可観測性基盤に何が流れるか”**も同じ**プライバシー境界の中で統制しなければならない**ことを意味します。 41

第四のギャップは、**DP/SMPC/TEE などの強い保証がまだ主流製品に入っていない**ことです。プライバシー保護 LLM のサーベイや AloePri、CMIF、LLM Access Shield のような研究は前進していますが、今回の主要実務製品群はそこへ到達していません。現状のベストプラクティスは、**境界を強くする・送る前に減らす・ロー**

カル実行を増やす・監査するであって、数学的プライバシー保証を既製品で買える段階ではない、というのが厳密な言い方です。 ⁴²

将来方向として有望だが、現時点では別カテゴリのもの

LLM Access Shield は、ドメイン適応型ポリシー enforcement、動的ポリシー customization、FPE による sensitive entity anonymization を組み合わせた研究フレームワークで、今回の「NBAccessに近いものを探す」という問いに対する学術的な近縁候補です。ただし、現時点では商用/OSS の汎用ゲートウェイや開発者 SDK として広く定着したというより、今後の設計方向を示す参照実装/研究成果として読むべきです。同様に AloePri や TEE+DP 系は、実務の主流ではなく、強い保証を求める次世代方向に位置づきます。 ⁴³

参考文献と一次情報

フレームワーク・製品ドキュメント

- NBAccess 公式リポジトリ、および関連実装の ClaudeRuntime / SourceVault。 ⁴⁴
- LiteLLM 公式 docs: getting started、providers、Ollama、OpenAI-compatible endpoints、model access groups、fallbacks、Presidio PII masking、enterprise。 ⁴⁵
- Haystack 公式 docs: routers、ConditionalRouter、TransformersTextRouter、LLMMessagesRouter、logging、tracing、Hayhooks、OpenAIChatGenerator、OllamaChatGenerator、FallbackChatGenerator。 ⁴⁶
- LangChain / LangGraph / LangSmith 公式 docs: middleware、guardrails、PII middleware、fallback middleware、Ollama integration、tracing、self-hosted LangSmith。 ⁴⁷
- Semantic Kernel 公式 docs と GitHub: overview、filters、supported languages、日本語ドキュメント、multi-service / serviceId / PromptExecutionSettings。 ¹⁴
- LlamaIndex 公式 docs: routers、using LLMs、local LLM tutorial、OpenAI / Ollama integrations、observability / instrumentation。 ⁴⁸
- Kong 公式 docs: AI Gateway、providers、Ollama provider、AI Proxy / AI Proxy Advanced、AI Prompt Guard、AI PII Sanitizer、audit log reference。 ⁴⁹
- Portkey 公式 docs: Configs、Universal API、custom_host、PII Redaction、audit logs、hybrid deployment、enterprise offering。 ⁵⁰
- Azure API Management AI Gateway 日本語公式 docs: genai gateway capabilities、backends、self-hosted gateway。 ¹⁸
- Dataiku LLM Mesh / DSS docs と日本語サイト: LLM Mesh、LLM connections、guardrails、NVIDIA NIM、Custom Dataiku / Cloud Stacks、trust and security。 ⁵¹

研究・セキュリティ指針

- NIST AI Risk Management Framework。 ⁵²
- OWASP Top 10 for Large Language Model Applications / OWASP GenAI Security Project。 ⁵³
- UK NCSC “Prompt injection is not SQL injection (it may be worse)” 。 ⁵⁴
- “Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution。” ⁵⁵
- “Privacy-Preserving Large Language Models: Mechanisms, Applications, and Challenges。” ⁵⁶
- “LLM Access Shield: Domain-Specific LLM Framework for Privacy Policy Compliance。” ⁵⁷
- “Towards Privacy-Preserving LLM Inference via Collaborative Obfuscation” (AloePri)。 ⁵⁸
- “Towards Confidential and Efficient LLM Inference with Dual ...” (CMIF, TEE+DP 系)。 ⁵⁹

総括

もし「NBAccessに似たもの」を厳密に探すなら、一般的なSDKやモデルルータをそのまま当てはめるのではなく、“機密の実体を **trusted side** に固定し、モデルには抽象化された表現だけを見せ、生成物の適用前に別の安全判定を通す”という設計原則を採るべきです。その原則に現実的に近づく組み合わせは、NBAccess的なアプリ内安全カーネルと、LiteLLM / Kong / Portkey / Azure APIM のような外部ゲートウェイの併用です。単独製品で万能なものはまだなく、強い **privacy boundary** は製品選定よりも、信頼境界をどこに置くかという設計判断で決まります。 60

1 3 6 10 24 27 28 GitHub - transreal/NBAccess · GitHub

<https://github.com/transreal/NBAccess>

2 16 49 AI Gateway - Kong Docs

https://developer.konghq.com/ai-gateway/?utm_source=chatgpt.com

4 43 57 Domain-Specific LLM Framework for Privacy Policy ...

https://arxiv.org/abs/2505.17145?utm_source=chatgpt.com

5 8 35 Fallbacks (Provider Failover)

https://docs.litellm.ai/docs/proxy/reliability?utm_source=chatgpt.com

7 37 52 AI Risk Management Framework | NIST

https://www.nist.gov/itl/ai-risk-management-framework?utm_source=chatgpt.com

9 20 32 40 60 GitHub - transreal/ClaudeRuntime: Expression-proposal loop state machine for Mathematica/Wolfram Language · GitHub

<https://github.com/transreal/ClaudeRuntime>

11 25 45 LiteLLM - Getting Started | liteLLM

https://docs.litellm.ai/?utm_source=chatgpt.com

12 30 ConditionalRouter - Haystack Documentation - Deepset AI

https://docs.haystack.deepset.ai/docs/conditionalrouter?utm_source=chatgpt.com

13 47 Overview - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/middleware/overview?utm_source=chatgpt.com

14 Introduction to Semantic Kernel

https://learn.microsoft.com/en-us/semantic-kernel/overview/?utm_source=chatgpt.com

15 48 Routers | Developer Documentation - LlamaParse

https://developers.llamaindex.ai/python/framework/module_guides/querying/router/?utm_source=chatgpt.com

17 26 50 Configs

https://docs.portkey.ai/docs/product/ai-gateway/configs?utm_source=chatgpt.com

18 Azure API Management の AI ゲートウェイ機能

https://learn.microsoft.com/ja-jp/azure/api-management/genai-gateway-capabilities?utm_source=chatgpt.com

19 51 One platform for enterprise AI success - Dataiku

https://www.dataiku.com/product?utm_source=chatgpt.com

21 Any OpenAI-Compatible Project

https://docs.portkey.ai/docs/integrations/libraries/openai-compatible?utm_source=chatgpt.com

22 Tracing

https://docs.haystack.deepset.ai/docs/2.29/tracing?utm_source=chatgpt.com

- 23 **Enterprise Offering**
https://docs.portkey.ai/docs/product/enterprise-offering?utm_source=chatgpt.com
- 29 **Model Access Groups**
https://docs.litellm.ai/docs/proxy/model_access_groups?utm_source=chatgpt.com
- 31 **Add chat completion services to Semantic Kernel**
https://learn.microsoft.com/en-us/semantic-kernel/concepts/ai-services/chat-completion/?utm_source=chatgpt.com
- 33 **AI PII Sanitizer - Plugin - Kong Docs**
https://developer.konghq.com/plugins/ai-sanitizer/?utm_source=chatgpt.com
- 34 **Router Retriever | Developer Documentation - LlamaParse**
https://developers.llamaindex.ai/python/framework/integrations/retrievers/router_retriever/?utm_source=chatgpt.com
- 36 **Tracing quickstart - Docs by LangChain**
https://docs.langchain.com/langsmith/observability-quickstart?utm_source=chatgpt.com
- 38 54 **Prompt injection is not SQL injection (it may be worse)**
https://www.ncsc.gov.uk/blog-post/prompt-injection-is-not-sql-injection?utm_source=chatgpt.com
- 39 55 **Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution**
https://arxiv.org/abs/2506.01055?utm_source=chatgpt.com
- 41 **Complete Logs Export**
https://docs.portkey.ai/docs/product/enterprise-offering/otel/complete-logs?utm_source=chatgpt.com
- 42 56 **Privacy-Preserving Large Language Models: Mechanisms ...**
https://arxiv.org/html/2412.06113v1?utm_source=chatgpt.com
- 44 **transreal/NBAccess**
https://github.com/transreal/NBAccess?utm_source=chatgpt.com
- 46 **Routers**
https://docs.haystack.deepset.ai/docs/routers?utm_source=chatgpt.com
- 53 **OWASP Top 10 for Large Language Model Applications**
https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com
- 58 **Towards Privacy-Preserving LLM Inference via Covariant ...**
https://arxiv.org/html/2603.01499v2?utm_source=chatgpt.com
- 59 **Towards Confidential and Efficient LLM Inference with Dual ...**
https://arxiv.org/abs/2509.09091?utm_source=chatgpt.com